Centro Oficial FP
Digital & Tech

SOLUCIÓN ACTIVIDAD EVALUABLE 6. BD-DAM. RA2. OBLIGATORIA.

RA2. Crea bases de datos definiendo su estructura y características de los elementos según el modelo relacional.

Criterios de evaluación

2.1 Analiza el formato de almacenamiento de la información.

2.2 Crea las tablas y las relaciones entre ellas.

2.3 Selecciona los tipos de datos adecuados.

2.4 Define los campos clave en las tablas.

2.5 Implanta las restricciones reflejadas en el diseño lógico.

2.6 Crea vistas.

2.7 Crea los usuarios y les asigna privilegios.

2.8 Utiliza asistentes, herramientas gráficas y los lenguajes de definición y control de datos.

TAREA – DISEÑO FÍSICO DE BASE DE DATOS (RA2) Unidad: Modelo físico – SQL (DDL) 1. Contexto de la tarea

Una tienda de informática necesita implantar una base de datos para gestionar su actividad diaria. La empresa vende productos informáticos, trabaja con proveedores, registra clientes, empleados, pedidos y líneas de pedido.

El diseño lógico ya lo han definido. Ahora debes transformarlo en un **modelo físico real**, utilizando **MySQL/MariaDB con XAMPP y phpMyAdmin**, creando la base de datos, las tablas, las relaciones, las restricciones y un pequeño conjunto de datos de prueba.

1. Creación de la base de datos

```
DROP DATABASE IF EXISTS tienda_informatica;

CREATE DATABASE tienda_informatica
CHARACTER SET utf8mb4
COLLATE utf8mb4_spanish_ci;

USE tienda_informatica;
```

2. Creación de tablas

```
CREATE TABLE proveedor (     nif
VARCHAR(9) PRIMARY KEY,
nombre VARCHAR(60) NOT NULL,
telefono VARCHAR(15),     email
VARCHAR(80) UNIQUE,     direccion
VARCHAR(120)
) ENGINE=InnoDB;

CREATE TABLE producto (
   codigo INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
nombre VARCHAR(80) NOT NULL,     marca VARCHAR(50)
NOT NULL,     precio_compra DECIMAL(8,2) NOT NULL,
precio_venta DECIMAL(8,2) NOT NULL,     stock INT NOT
NULL DEFAULT 0,
   CONSTRAINT chk_precio_compra CHECK (precio_compra > 0),
```

```
    CONSTRAINT chk_precio_venta CHECK (precio_venta > 0),
    CONSTRAINT chk_stock CHECK (stock >= 0)
) ENGINE=InnoDB;

CREATE TABLE cliente (
    id_cliente INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
nombre VARCHAR(60) NOT NULL,     telefono VARCHAR(15),
email VARCHAR(80) UNIQUE,     direccion VARCHAR(120)
) ENGINE=InnoDB;

CREATE TABLE empleado (
    id_empleado INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
nombre VARCHAR(60) NOT NULL,     telefono VARCHAR(15),
puesto VARCHAR(40) NOT NULL
) ENGINE=InnoDB;

CREATE TABLE pedido (
    id_pedido INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    fecha DATE NOT NULL,     total
DECIMAL(10,2) NOT NULL,     iva_aplicado
DECIMAL(5,2) NOT NULL,     id_cliente INT
UNSIGNED NOT NULL,     id_empleado INT
UNSIGNED NOT NULL,     CONSTRAINT
chk_total CHECK (total >= 0),
    CONSTRAINT chk_iva CHECK (iva_aplicado >= 0),
    CONSTRAINT fk_pedido_cliente
        FOREIGN KEY (id_cliente)
        REFERENCES cliente(id_cliente)
        ON DELETE RESTRICT
        ON UPDATE CASCADE,
    CONSTRAINT fk_pedido_empleado
        FOREIGN KEY (id_empleado)
        REFERENCES empleado(id_empleado)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
) ENGINE=InnoDB;
```

```
CREATE TABLE linea_pedido (     id_pedido INT
UNSIGNED NOT NULL,     codigo_producto INT
UNSIGNED NOT NULL,
   cantidad  INT  NOT  NULL,       precio_unitario
DECIMAL(8,2)  NOT  NULL,          PRIMARY  KEY
(id_pedido, codigo_producto),
   CONSTRAINT chk_cantidad CHECK (cantidad > 0),
   CONSTRAINT chk_precio_unitario CHECK (precio_unitario > 0),
   CONSTRAINT fk_linea_pedido
      FOREIGN KEY (id_pedido)
      REFERENCES pedido(id_pedido)
      ON DELETE CASCADE
      ON UPDATE CASCADE,
   CONSTRAINT fk_linea_producto
      FOREIGN KEY (codigo_producto)
      REFERENCES producto(codigo)
      ON DELETE RESTRICT
      ON UPDATE CASCADE
) ENGINE=InnoDB;

CREATE TABLE suministro (     nif_proveedor
VARCHAR(9) NOT NULL,     codigo_producto
INT UNSIGNED NOT NULL,
precio_suministro DECIMAL(8,2) NOT NULL,
plazo_entrega INT NOT NULL,
   PRIMARY KEY (nif_proveedor, codigo_producto),
   CONSTRAINT chk_precio_suministro CHECK (precio_suministro > 0),
   CONSTRAINT chk_plazo CHECK (plazo_entrega >= 0),
   CONSTRAINT fk_suministro_proveedor
      FOREIGN KEY (nif_proveedor)
      REFERENCES proveedor(nif)
      ON DELETE RESTRICT
      ON UPDATE CASCADE,
   CONSTRAINT fk_suministro_producto
      FOREIGN KEY (codigo_producto)
      REFERENCES producto(codigo)
      ON DELETE RESTRICT
      ON UPDATE CASCADE )
ENGINE=InnoDB;
```

3. Explicación de claves primarias

Las claves primarias simples son:

- proveedor.nif
- producto.codigo
- cliente.id_cliente
- empleado.id_empleado
- pedido.id_pedido

Las claves primarias compuestas son:

- linea_pedido (id_pedido, codigo_producto)
- suministro (nif_proveedor, codigo_producto)

Se usan claves compuestas porque una línea de pedido se identifica por la combinación de pedido y producto, y un suministro se identifica por la combinación de proveedor y producto.

4. Justificación de las claves foráneas

- En pedido, las claves foráneas a cliente y empleado usan:
    - ON DELETE RESTRICT para impedir borrar clientes o empleados con pedidos asociados
    - ON UPDATE CASCADE para propagar cambios de clave
- En linea_pedido, la clave foránea a pedido usa:
    - ON DELETE CASCADE porque si se elimina un pedido, deben borrarse sus líneas
- En linea_pedido, la clave foránea a producto usa:
    - ON DELETE RESTRICT para no eliminar productos presentes en pedidos históricos
- En suministro, ambas claves usan RESTRICT para proteger relaciones existentes

5. Datos de prueba

```
INSERT INTO proveedor (nif, nombre, telefono, email, direccion) VALUES
('A12345678', 'Tech Distribuciones', '910000001', 'contacto@techdist.es', 'Calle Mayor 10, Madrid'),
('B23456789', 'MicroComponentes SL', '910000002', 'ventas@microcomp.es', 'Avenida Europa 20, Valencia'),
('C34567890', 'Hardware Global', '910000003', 'info@hardwareglobal.es', 'Gran Vía 15, Barcelona');

INSERT INTO producto (nombre, marca, precio_compra, precio_venta, stock) VALUES
('Portátil 15 pulgadas', 'Lenovo', 450.00, 599.00, 12),
('Monitor 24 pulgadas', 'Samsung', 110.00, 159.00, 20),
('Teclado mecánico', 'Logitech', 35.00, 59.00, 30),
('Ratón inalámbrico', 'HP', 12.00, 25.00, 50),
('Disco SSD 1TB', 'Kingston', 60.00, 95.00, 18);

INSERT INTO cliente (nombre, telefono, email, direccion) VALUES
('Ana López', '600111222', 'ana@correo.com', 'Calle Sol 3, Sevilla'),
('Luis Pérez', '600333444', 'luis@correo.com', 'Calle Luna 7, Málaga'),
('Marta Ruiz', '600555666', 'marta@correo.com', 'Calle Río 12, Córdoba');

INSERT INTO empleado (nombre, telefono, puesto) VALUES
('Carlos Gómez', '611111111', 'Vendedor'),
('Laura Díaz', '622222222', 'Encargada');

INSERT INTO pedido (fecha, total, iva_aplicado, id_cliente, id_empleado) VALUES
('2025-02-01', 684.00, 21.00, 1, 1),
('2025-02-03', 184.00, 21.00, 2, 2),
('2025-02-05', 120.00, 21.00, 3, 1);

INSERT INTO linea_pedido (id_pedido, codigo_producto, cantidad, precio_unitario) VALUES
(1, 1, 1, 599.00),
(1, 3, 1, 59.00),
(2, 2, 1, 159.00),
(2, 4, 1, 25.00),
(3, 5, 1, 95.00),
(3, 4, 1, 25.00);

INSERT INTO suministro (nif_proveedor, codigo_producto, precio_suministro, plazo_entrega) VALUES
('A12345678', 1, 430.00, 5),
```

('A12345678', 2, 105.00, 4),
('B23456789', 3, 32.00, 3),
('C34567890', 5, 58.00, 6);

6. Consultas de comprobación

SELECT * FROM proveedor;
SELECT * FROM producto;
SELECT * FROM cliente;
SELECT * FROM empleado;
SELECT * FROM pedido;
SELECT * FROM linea_pedido; SELECT
* FROM suministro;

Consulta con varias tablas:

```
SELECT
   p.id_pedido,
   c.nombre AS cliente,
   e.nombre AS empleado,
   p.fecha,
   p.total FROM
pedido p
INNER JOIN cliente c ON p.id_cliente = c.id_cliente INNER
JOIN empleado e ON p.id_empleado = e.id_empleado;
```

Consulta con líneas:

```
SELECT
   lp.id_pedido,
pr.nombre AS producto,
lp.cantidad,
lp.precio_unitario
FROM linea_pedido lp
INNER JOIN producto pr ON lp.codigo_producto = pr.codigo;
```

7. Pruebas de integridad a) Producto con precio de venta negativo

Debe fallar.

INSERT INTO producto (nombre, marca, precio_compra, precio_venta, stock) VALUES ('Impresora básica', 'Canon', 40.00, -10.00, 5);

b) Línea de pedido con producto inexistente

Debe fallar por clave foránea.

INSERT INTO linea_pedido (id_pedido, codigo_producto, cantidad, precio_unitario) VALUES (1, 999, 1, 20.00);

c) Borrar cliente con pedidos asociados

Debe fallar por ON DELETE RESTRICT. DELETE

FROM cliente WHERE id_cliente = 1;

d) Insertar proveedor con NIF repetido

Debe fallar por clave primaria.

INSERT INTO proveedor (nif, nombre, telefono, email, direccion)
VALUES ('A12345678', 'Proveedor repetido', '900000000', 'rep@correo.com', 'Dirección inventada');

8. Uso de herramientas gráficas

Se debe utilizar phpMyAdmin para:

- Crear al menos una tabla
- Ver la estructura
- Revisar relaciones
- Exportar la base de datos

Ejemplo de explicación para entregar:

Se ha utilizado phpMyAdmin para crear la base de datos y revisar las tablas. También se ha comprobado la estructura de campos, claves primarias y claves foráneas. Las relaciones se han verificado desde la vista de estructura de cada tabla.

9. Modificaciones del esquema con ALTER TABLE Añadir descripcion a producto

```
ALTER TABLE producto
ADD descripcion VARCHAR(200);
```

Modificar tamaño del campo nombre en cliente

```
ALTER TABLE cliente
MODIFY nombre VARCHAR(80) NOT NULL;
```

Añadir fecha_alta en cliente con valor por defecto

```
ALTER TABLE cliente
ADD fecha_alta DATETIME DEFAULT CURRENT_TIMESTAMP;
```

Eliminar telefono de empleado

```
ALTER TABLE empleado
DROP COLUMN telefono;
```

10. Estructura final comprobada

```
DESCRIBE proveedor;
DESCRIBE producto;
DESCRIBE cliente;
DESCRIBE empleado;
DESCRIBE pedido;
DESCRIBE linea_pedido; DESCRIBE
suministro;
```

11. Diccionario de datos Tabla: proveedor

Campo	Tipo	Restricciones	Descripción
nif	VARCHAR(9)	PK	Identificador fiscal del proveedor
nombre	VARCHAR(60)	NOT NULL	Nombre del proveedor
telefono	VARCHAR(15)	—	Teléfono de contacto
email	VARCHAR(80)	UNIQUE	Correo electrónico
direccion	VARCHAR(120)	—	Dirección postal

Tabla: producto

Campo	Tipo	Restricciones	Descripción
codigo	INT UNSIGNED	PK, AUTO_INCREMENT	Identificador del producto
nombre	VARCHAR(80)	NOT NULL	Nombre comercial
marca	VARCHAR(50)	NOT NULL	Marca del producto
precio_compra	DECIMAL(8,2)	NOT NULL, CHECK > 0	Precio al proveedor
precio_venta	DECIMAL(8,2)	NOT NULL, CHECK > 0	Precio de venta
stock	INT	NOT NULL, DEFAULT 0, CHECK >= 0	Unidades disponibles
descripcion	VARCHAR(200)	—	Descripción del producto

Tabla: cliente

Campo	Tipo	Restricciones	Descripción
id_cliente	INT UNSIGNED	PK, AUTO_INCREMENT	Identificador del cliente
nombre	VARCHAR(80)	NOT NULL	Nombre del cliente
telefono	VARCHAR(15)	—	Teléfono
email	VARCHAR(80)	UNIQUE	Correo electrónico
direccion	VARCHAR(120)	—	Dirección
fecha_alta	DATETIME	DEFAULT CURRENT_TIMESTAMP	Fecha de alta

Tabla: empleado

Campo	Tipo	Restricciones	Descripción
id_empleado	INT UNSIGNED	PK, AUTO_INCREMENT	Identificador del empleado
nombre	VARCHAR(60)	NOT NULL	Nombre completo
puesto	VARCHAR(40)	NOT NULL	Cargo en la tienda

Tabla: pedido

Campo	Tipo	Restricciones	Descripción
id_pedido	INT UNSIGNED	PK, AUTO_INCREMENT	Identificador del pedido
fecha	DATE	NOT NULL	Fecha del pedido
total	DECIMAL(10,2)	NOT NULL, CHECK >= 0	Importe total
iva_aplicado	DECIMAL(5,2)	NOT NULL, CHECK >= 0	IVA aplicado
id_cliente	INT UNSIGNED	FK	Cliente que realiza el pedido
id_empleado	INT UNSIGNED	FK	Empleado que atiende

Tabla: linea_pedido

Campo	Tipo	Restricciones	Descripción
id_pedido	INT UNSIGNED	PK, FK	Pedido al que pertenece
codigo_producto	INT UNSIGNED	PK, FK	Producto incluido
cantidad	INT	NOT NULL, CHECK > 0	Unidades vendidas
precio_unitario	DECIMAL(8,2)	NOT NULL, CHECK > 0	Precio por unidad

Tabla: suministro

Campo	Tipo	Restricciones	Descripción
nif_proveedor	VARCHAR(9)	PK, FK	Proveedor relacionado
codigo_producto	INT UNSIGNED	PK, FK	Producto suministrado
precio_suministro	DECIMAL(8,2)	NOT NULL, CHECK > 0	Precio de compra acordado
plazo_entrega	INT	NOT NULL, CHECK >= 0	Días de entrega

ACTIVIDAD 10

En esta actividad se crean varios usuarios con distintos privilegios sobre la base de datos tienda_informatica.
La idea es que no todos los usuarios tengan el mismo nivel de acceso:

- un usuario solo podrá **consultar**
- otro podrá **consultar e insertar**
- otro tendrá **control total**

1. Crear los usuarios

```
CREATE USER 'consulta_tienda'@'localhost' IDENTIFIED BY 'consulta123';
CREATE USER 'gestion_tienda'@'localhost' IDENTIFIED BY 'gestion123';
CREATE USER 'admin_tienda'@'localhost' IDENTIFIED BY 'admin123';
```

2. Asignar privilegios

Usuario 1: solo consulta

```
GRANT SELECT ON tienda_informatica.* TO 'consulta_tienda'@'localhost';
```

Usuario 2: consulta e inserción

```
GRANT SELECT, INSERT ON tienda_informatica.* TO 'gestion_tienda'@'localhost';
```

Usuario 3: todos los privilegios

```
GRANT ALL PRIVILEGES ON tienda_informatica.* TO 'admin_tienda'@'localhost';
```

Aplicar cambios

```
FLUSH PRIVILEGES;
```

3. Comprobación de privilegios

```
SHOW GRANTS FOR 'consulta_tienda'@'localhost';
SHOW GRANTS FOR 'gestion_tienda'@'localhost';
SHOW GRANTS FOR 'admin_tienda'@'localhost';
```

4. Pruebas de funcionamiento

Prueba 1 – Usuario consulta_tienda

Debe poder consultar, pero no insertar, borrar ni modificar.

Consulta válida

```
SELECT * FROM cliente;
```

Operación que debe fallar

```
INSERT INTO cliente (nombre, telefono, email, direccion)
VALUES ('Carlos Soto', '600777888', 'carlos@correo.com', 'Calle Nueva 5');
```

Resultado esperado

La consulta funciona, pero el INSERT debe fallar por falta de privilegios.

Prueba 2 – Usuario gestion_tienda

Debe poder consultar e insertar, pero no borrar ni modificar estructura.

Consulta válida

```
SELECT * FROM producto;
```

Inserción válida

```
INSERT INTO cliente (nombre, telefono, email, direccion)
VALUES ('Carlos Soto', '600777888', 'carlos@correo.com', 'Calle Nueva 5');
```

Operación que debe fallar

```
DELETE FROM cliente WHERE id_cliente = 1;
```

Resultado esperado

- SELECT funciona
- INSERT funciona
- DELETE falla

Prueba 3 – Usuario admin_tienda

Debe poder realizar cualquier operación.

Consulta válida

```
SELECT * FROM pedido;
```

Inserción válida

```
INSERT INTO producto (nombre, marca, precio_compra, precio_venta, stock)
VALUES ('Webcam HD', 'Logitech', 20.00, 39.00, 10);
```

Modificación válida

```
UPDATE producto
SET stock = 15
WHERE codigo = 1;
```

Cambio estructural válido

```
ALTER TABLE cliente ADD observaciones VARCHAR(100);
```

Resultado esperado
Todas las operaciones deben funcionar.

5. Revocar privilegios
Si se desea quitar permisos al usuario gestion_tienda:

```
REVOKE INSERT ON tienda_informatica.* FROM 'gestion_tienda'@'localhost';
FLUSH PRIVILEGES;
```

6. Eliminar usuarios

```
DROP USER 'consulta_tienda'@'localhost';
DROP USER 'gestion_tienda'@'localhost';
DROP USER 'admin_tienda'@'localhost';
```

7. Explicación para entregar
Usuario consulta_tienda
Solo puede leer datos mediante SELECT.
Usuario gestion_tienda
Puede leer e insertar información nueva, pero no borrar ni modificar la estructura.
Usuario admin_tienda
Tiene control total sobre la base de datos y puede ejecutar cualquier operación.

Actividad 11. Creación de vistas
Solución
Las vistas son **tablas virtuales** basadas en una consulta. No almacenan datos por separado, sino que muestran el resultado de un SELECT.
Sirven para:

- simplificar consultas complejas
- ocultar columnas innecesarias
- mostrar solo parte de la información
- mejorar seguridad

1. Crear una vista simple
Enunciado resuelto
Crear una vista que muestre los pedidos con el nombre del cliente y del empleado.

```
CREATE VIEW vista_pedidos_clientes AS
SELECT
   p.id_pedido,
   c.nombre AS cliente,
   e.nombre AS empleado,
   p.fecha,
   p.total
FROM pedido p
```

```
INNER JOIN cliente c ON p.id_cliente = c.id_cliente
INNER JOIN empleado e ON p.id_empleado = e.id_empleado;
```

2. Usar la vista

```
SELECT * FROM vista_pedidos_clientes;
```

3. Crear otra vista con líneas de pedido

```
CREATE VIEW vista_lineas_pedido AS
SELECT
   lp.id_pedido,
   pr.nombre AS producto,
   lp.cantidad,
   lp.precio_unitario
FROM linea_pedido lp
INNER JOIN producto pr ON lp.codigo_producto = pr.codigo;
```

Consulta de la vista

```
SELECT * FROM vista_lineas_pedido;
```

4. Vista con filtro

Crear una vista que muestre solo productos con stock superior a 10.

```
CREATE VIEW vista_productos_stock AS
SELECT
   codigo,
   nombre,
   marca,
   stock
FROM producto
WHERE stock > 10;
```

Usar la vista

```
SELECT * FROM vista_productos_stock;
```

5. Eliminar una vista

```
DROP VIEW vista_productos_stock;
```

6. Explicación para entregar

¿Qué es una vista?

Una vista es una consulta guardada que se utiliza como si fuera una tabla.

¿Para qué sirve?

- Para simplificar consultas largas
- Para mostrar solo ciertos campos
- Para restringir información visible
- Para reutilizar consultas frecuentes

Vista creada en la actividad

Se ha creado la vista vista_pedidos_clientes, que muestra cada pedido con el nombre del cliente y el empleado que lo atendió.

Consulta sobre la vista

Después se ha utilizado:

SELECT * FROM vista_pedidos_clientes;

Esto permite consultar la información sin repetir el JOIN.

Resumen final de ambas actividades

Usuarios creados

```
CREATE USER 'consulta_tienda'@'localhost' IDENTIFIED BY 'consulta123';
CREATE USER 'gestion_tienda'@'localhost' IDENTIFIED BY 'gestion123';
CREATE USER 'admin_tienda'@'localhost' IDENTIFIED BY 'admin123';

GRANT SELECT ON tienda_informatica.* TO 'consulta_tienda'@'localhost';
GRANT SELECT, INSERT ON tienda_informatica.* TO 'gestion_tienda'@'localhost';
GRANT ALL PRIVILEGES ON tienda_informatica.* TO 'admin_tienda'@'localhost';

FLUSH PRIVILEGES;
```

Vista creada

```
CREATE VIEW vista_pedidos_clientes AS
SELECT
    p.id_pedido,
    c.nombre AS cliente,
    e.nombre AS empleado,
    p.fecha,
    p.total
FROM pedido p
INNER JOIN cliente c ON p.id_cliente = c.id_cliente
INNER JOIN empleado e ON p.id_empleado = e.id_empleado;
```

Uso de la vista

```
SELECT * FROM vista_pedidos_clientes;
```